JavaScript is disabled on your browser.



compact1, compact2, compact3

java.util

# Class ConcurrentModificationException

java.lang.Object
  java.lang.Throwable
    java.lang.Exception
      java.lang.RuntimeException
        java.util.ConcurrentModificationException

**All Implemented Interfaces:**

Serializable

**Direct Known Subclasses:**

DirectoryIteratorException

---

```
public class ConcurrentModificationException
extends RuntimeException
```

This exception may be thrown by methods that have detected concurrent modification of an object when such modification is not permissible.

For example, it is not generally permissible for one thread to modify a Collection while another thread is iterating over it. In general, the results of the iteration are undefined under these circumstances. Some Iterator implementations (including those of all the general purpose collection implementations provided by the JRE) may choose to throw this exception if this behavior is detected. Iterators that do this are known as *fail-fast* iterators, as they fail quickly and cleanly, rather that risking arbitrary, non-deterministic behavior at an undetermined time in the future.

Note that this exception does not always indicate that an object has been concurrently modified by a *different* thread. If a single thread issues a sequence of method invocations that violates the contract of an object, the object may throw this exception. For example, if a thread modifies a collection directly while it is iterating over the collection with a fail-fast iterator, the iterator will throw this exception.

Note that fail-fast behavior cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast operations throw `ConcurrentModificationException` on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *`ConcurrentModificationException` should be used only to detect bugs.*

**Since:**

1.2

**See Also:**

`Collection`, `Iterator`, `Spliterator`, `ListIterator`, `Vector`, `LinkedList`, `HashSet`, `Hashtable`, `TreeMap`, `AbstractList`, Serialized Form

## *Constructor Summary*

**Constructors**

| Constructor | Description |
| --- | --- |
| `ConcurrentModificationException()` | Constructs a ConcurrentModif with no detail me |
| `ConcurrentModificationException(String message)` | Constructs a `ConcurrentModif` with the specified |
| `ConcurrentModificationException(String message, Throwable cause)` | Constructs a new specified detail m |
| `ConcurrentModificationException(Throwable cause)` | Constructs a new specified cause a message of `(caus` `cause.toString(` contains the class message of `cause` |

## *Method Summary*

### Methods inherited from class java.lang.Throwable

`addSuppressed`, `fillInStackTrace`, `getCause`, `getLocalizedMessage`, `getMessage`, `getStackTrace`, `getSuppressed`, `initCause`, `printStackTrace`, `printStackTrace`, `printStackTrace`, `setStackTrace`, `toString`

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

## *Constructor Detail*

### ConcurrentModificationException

```
public ConcurrentModificationException()
```

Constructs a ConcurrentModificationException with no detail message.

### ConcurrentModificationException

```
public ConcurrentModificationException(String message)
```

Constructs a `ConcurrentModificationException` with the specified detail message.

**Parameters:**

`message - the detail message pertaining to this exception.`

### ConcurrentModificationException

```
public ConcurrentModificationException(Throwable cause)
```

Constructs a new exception with the specified cause and a detail message of `(cause==null ? null : cause.toString())` (which typically contains the class and detail message of `cause`.

**Parameters:**

`cause - the cause (which is saved for later retrieval by the Throwable.getCause() method). (A null value is permitted, and indicates that the cause is nonexistent or unknown.)`

**Since:**

`1.7`

### ConcurrentModificationException

```
public ConcurrentModificationException(String message,
                                       Throwable cause)
```

Constructs a new exception with the specified detail message and cause.

Note that the detail message associated with `cause` is *not* automatically incorporated in this exception's detail message.

**Parameters:**

`message - the detail message (which is saved for later retrieval by the Throwable.getMessage() method).`

`cause - the cause (which is saved for later retrieval by the Throwable.getCause() method). (A null value is permitted, and indicates that the cause is nonexistent or unknown.)`

**Since:**

1.7




Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.